

AI 博客平台部署架构与请求链路

基于仓库实现整理 · 编排: `docker/compose.prod.yml` · 入口配置: `docker/nginx/*.conf` · 2026-09-23

一、总览：一台服务器跑了什么

腾讯云 ECS (81.71.65.154, 2C4G) 上用 docker compose 跑 8 个服务。容器之间通过 compose 内置 DNS 按**服务名**互访；对公网只开放入口 nginx 的端口，其余容器一律不发布。

服务名	容器名	角色	容器内监听	发布到公网
nginx	blog-nginx	统一入口（反向代理）	80 / 443 / 8081 / 8082	80、443、8081、8082
blog	blog-blog	前台博客 (Next.js standalone)	3000	不发布
server	blog-server	Go 后端 (API + Worker)	8080	不发布
admin	blog-admin	管理端 (nginx:alpine 托管静态)	80	不发布
mysql / redis / rabbitmq / qdrant	blog-*	中间件 ×4	3306 / 6379 / 5672 / 6333	不发布 (rabbitmq 管理界面仅绑回环，走 SSH 隧道)

核心心智模型：**compose 的 ports：管“大门开不开”（宿主机发布），容器内监听哪个端口是各镜像自己的事。**容器之间互访永远走内部端口，与宿主机映射无关。

二、两层 nginx 与三份 conf 文件

整个系统有**两层 nginx**，职责不同：

层	配置在哪	职责
入口层 (blog-nginx 容器)	<code>docker/nginx/blog.conf</code> 、 <code>admin.conf</code> 、 <code>default.conf</code> ，按文件名挂载进 <code>conf.d/</code>	公网大门：按域名/端口选 server 块、按路径选 location、SSE 专项配置、444 兜底
容器内层 (仅 admin 镜像自带)	<code>docker/Dockerfile.admin</code> 构建时写入容器 <code>conf.d</code>	SPA 路由回退、带哈希静态资源长缓存、容器内 <code>/api</code> 反代到 <code>server:8080</code>

注意同名陷阱：`docker/nginx/admin.conf`（入口层）与 `Dockerfile.admin` 里写进容器的 `admin.conf`（容器内层）**同名不同物**。blog 容器没有内层 nginx——Next.js standalone 直接跑 node。

入口层拆三份文件的理由：nginx 按 `conf.d` 文件加载，**一个文件一个 server 块、一个职责**，增删子域互不影响，compose 挂载与 CI 同步都按文件粒度走。

文件	管什么	要点
blog.conf	前台博客 ljl0628.cn / www (+ 备案期 8081 预览块)	唯一按路径分流的块： / → blog:3000, /api → server:8080, 问答接口有 SSE 专线
admin.conf	管理端 admin.ljl0628.cn (+ 备案期 8082 预览块)	整站一把转给 admin:80, 路径分流交给容器内层
default.conf	兜底：IP 直连 / 陌生域名	return 444 直接断开。没有它，未知 Host 会落到字母序第一个块 (admin)，裸 IP 就能打开管理端登录页

三、域名、DNS 与备案期预览

admin.ljl0628.cn 是**子域名**：注册了主域名即拥有其全部子域名，无需另外申请，只需在 DNS 解析里加 A 记录。备案针对主域名，子域名随主域名，不单独备案。备案通过后计划加三条记录，全部指向同一台服务器：

主机记录	记录值	作用
@	81.71.65.154	ljl0628.cn → 博客
www		www.ljl0628.cn → 博客
admin		admin.ljl0628.cn → 管理端

浏览器默认端口：http:// 默认 80、https:// 默认 443，地址栏隐藏默认端口；写了端口（如 :8081）则用写的。备案期间没有域名解析，用 IP:端口 直接访问预览站（8081 = 博客、8082 = 管理端），备案通过后随预览块一起删除。

DNS 与 nginx 的分工：**DNS 负责把所有域名送到服务器门口，nginx 的 server_name 负责在门口按域名分流。**

四、nginx 怎么选 server 块：先端口，后 Host

conf.d 里多个 server 块并存，nginx 两步选出处理请求的块：

步骤	依据	规则
① 按端口筛	请求连的端口	只在 监听该端口 的 server 块里挑，其他块不参与
② 按 Host 挑	请求头 Host	与各块 server_name 精确匹配；都不匹配 → 该端口的 default_server（或该端口读入的第一个块）

拿当前配置套用：

访问	① 端口	② Host	结果
ljl0628.cn/xxx	80（三块候选）	匹配 blog 主块	博客主块处理
admin.ljl0628.cn	80	匹配 admin 块	管理端块处理
81.71.65.154（IP 直连）	80	是 IP，全不匹配	default.conf → 444 断开
81.71.65.154:8081	8081（唯一监听块）	走不到这步	博客预览块必进

预览块**不写 server_name**：备案期用 IP:端口 访问，Host 是 IP，写什么域名都匹配不上；不写，它自动成为该端口的默认块，来者不拒。预览块与主块的隔离手段是**端口**，不是域名。

五、blog 主块怎么选 location：精确匹配优先，前缀取最长

```
location = /api/v1/portal/ask { ... } # 精确匹配，优先级最高，SSE 专线
location /api/ { ... } # 前缀匹配：其余 API
location / { ... } # 前缀匹配兜底：页面与静态资源 → Next.js
```

请求路径	命中	上游
POST /api/v1/portal/ask	= /api/v1/portal/ask	server:8080（SSE 专项配置）
/api/v1/articles、/portal/figures/x.svg ...	/api/	server:8080（60s 读超时）
/、/article/xxx、/_next/static/...	/	blog:3000

SSE 专线多出的四行配置（2026-09-23 生产 8081 预览不流式的根因就是漏了这段）：

配置	作用
proxy_buffering off	关键 ：nginx 默认把上游响应攒进缓冲区再大块吐出，SSE 的逐字小包会被攒成一坨，前端憋到结束才见字
proxy_cache off	流式响应不进缓存
proxy_set_header Connection ""	对上游保持干净的 HTTP/1.1 长连接
proxy_read_timeout 300s	默认 60s 会掐断长回答中途的停顿，拉长到 300s

六、proxy_pass 的目标：服务名 + 容器内部端口

proxy_pass http://server:8080 里的 server 是 compose 的**服务名**（services: 下的键），不是容器名（blog-server）。Docker 内置 DNS 两者都能解析，但约定一律写服务名——服务名是“逻辑角色”的稳定称呼，容器改名/重建不受影响。

写法	服务名	内部端口由谁决定
<code>http://blog:3000</code>	blog	Dockerfile.blog 的 <code>ENV PORT=3000 HOSTNAME=0.0.0.0</code> (Next 读它; <code>HOSTNAME=0.0.0.0</code> 不设会只绑 127.0.0.1, 容器外连不进)
<code>http://server:8080</code>	server	<code>config.yaml</code> 的 <code>server.port: 8080</code>
<code>http://admin:80</code>	admin	镜像内层 nginx 的 <code>listen 80</code>

与端口相关的三个易混概念:

概念	在哪定义	作用
容器内监听端口	进程自身配置 (ENV / config / 容器内 conf)	进程真正绑定的端口, 内网邻居用"服务名:它"访问
<code>EXPOSE 3000</code>	Dockerfile	纯文档声明 , 不开放/发布任何端口; 删掉不影响连通性。只影响 <code>docker run -P</code> 和 <code>docker ps</code> 显示
<code>ports: 8081:8081</code>	compose	把容器端口发布到宿主机 (大门)。宿主机口与内部口可以不同 (如本地 dev 的 3309:3306) ; 不配 = 没有门

七、四条典型请求链路走查

7.1 博客首页 `http://lj10628.cn/`

1. 浏览器查**公网 DNS**: `lj10628.cn` → `81.71.65.154`, 默认补 `:80` 发起请求, 请求头 `Host: lj10628.cn`
2. 到达 `blog-nginx` 容器 (`compose 80:80` 发布) ; ① 端口 80 → ② Host 命中 `blog` 主块
3. 路径 / 命中 `location /` → `proxy_pass http://blog:3000`
4. `blog` 容器内 `Next.js` (服务端取数走内网直连 `server:8080`) 渲染返回, `nginx` 原样转回浏览器

7.2 RAG 问答 `POST /api/v1/portal/ask`

1. 同上前两步进入 `blog` 主块; 路径**精确匹配** `SSE` 专线 `location`
2. `proxy_buffering off` 下, `Go` 后端每写一帧 `SSE` 就刷出, `nginx` 不攒包, 前端逐字渲染
3. `LLM` 生成期间连接保持, 最长等 `300s`; 回答完毕连接结束

7.3 管理端 `http://admin.lj10628.cn/` (两层 `nginx`)

1. `DNS` → `81.71.65.154:80`; Host 命中 `admin` 块 → **整站**转给 `admin:80`
2. `admin` 容器的**内层 nginx** 接手: `/assets/` 长缓存、/ `SPA` 回退 `try_files $uri /index.html`、`/api/` 再反代 `server:8080`
3. 浏览器全程只认 `admin.lj10628.cn`, 内层分流对它不可见

7.4 备案期预览 `http://81.71.65.154:8081/`

1. 浏览器直连 IP, 显式端口 `8081` (非默认端口必须手写)
2. ① 端口筛选: `8081` 上只有博客预览块, **必进**, 与 Host 无关 (预览块无 `server_name`)

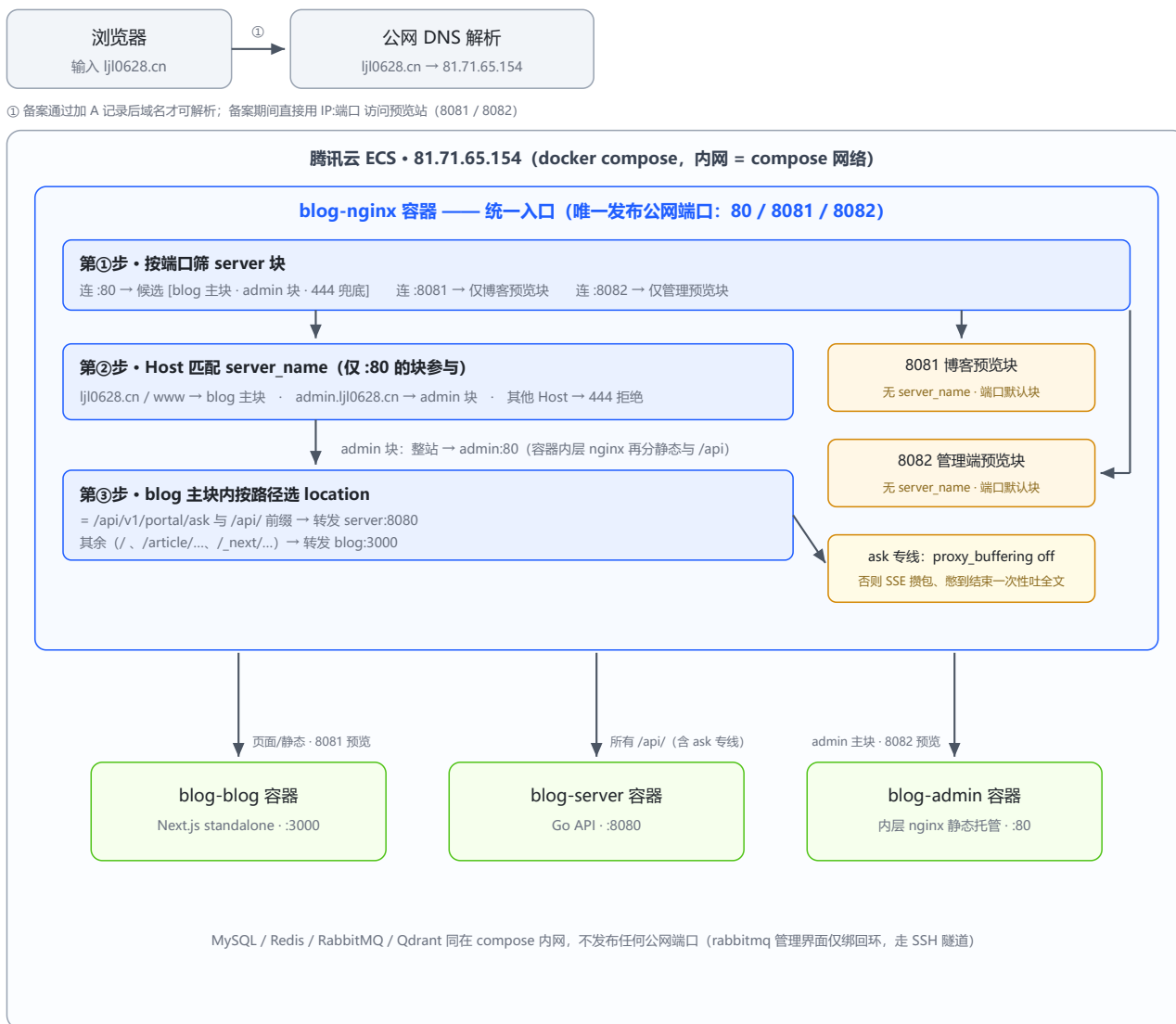
3. `location /` → `blog:3000` 出页面; `location = /api/v1/portal/ask` → SSE 专线 (2026-09-23 补配并验证)

两次独立寻址: 浏览器查**公网 DNS** 得到服务器 IP; nginx 转发时查的是 **Docker 内网 DNS** (服务名 → 容器 IP)。 `http://admin:80` 这类地址只存在于服务器内部, 公网不可达。

八、FAQ 速查

问题	一句话答案
为什么三份 nginx conf?	一个文件一个 server 块/一个职责: blog 分流、admin 整转、default 兜底防 IP 直连扫描
怎么知道执行哪个 server 块?	先按端口筛 (8081 上唯一块必进), 再按 Host 匹配 server_name, 都不中 → default_server
<code>proxy_pass</code> 里的 server 是容器名吗?	是 compose 服务名 (容器名叫 blog-server); 写服务名不写容器名, 稳定
<code>EXPOSE 3000</code> 是开放端口吗?	不是, 纯文档声明; 真正让进程听 3000 的是 <code>ENV PORT=3000</code> , 让外网能访问的是 <code>compose ports</code> :
内部端口 vs 发布端口?	内部端口 = 容器里进程监听的口 (内网邻居用); 发布端口 = <code>宿主机:容器</code> 映射, 给外部调用者开的门, 可以不配
浏览器只输域名访问哪个端口?	http 默认 80、https 默认 443, 地址栏隐藏默认端口; 写 :8081 就用 8081
admin.ljl0628.cn 哪来的?	自定义子域名, 注册主域名即拥有; 备案后加 A 记录指向服务器即可用
问答为什么不流式 (排查口诀)?	查两条: ①nginx 有没有 <code>location = /api/v1/portal/ask</code> 且 <code>proxy_buffering off</code> ; ②本地 dev 是否经 Next rewrites (gzip 会缓冲 SSE, 须直连后端)
备案通过后删什么?	两个 conf 的预览块 + compose 的 8081/8082 端口映射; 加 DNS 记录、放开 443

九、流程图：一次请求的完整判定链路



图：一次请求的判定链路 —— 端口定块 → Host 定块 → 路径定 location → proxy_pass 进容器；浏览器查公网 DNS，nginx 查 Docker 内网 DNS（两次独立寻址）